History of programming languages and paradigms

Presented by:

Cristian Camilo Cortes Ortiz

Codecamp academy

Table of contents

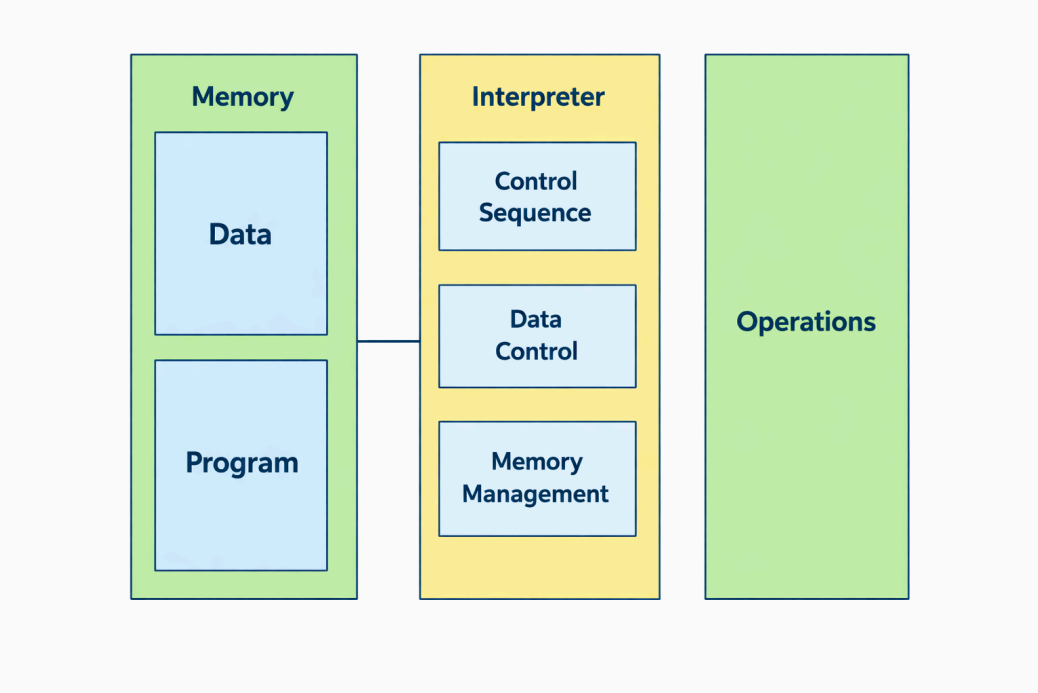
Abstract machine 4
Execution cycle 5
Programming languages vs natural languages 6
Levels of description 7
Grammar 7
Semantics 7
Pragmatics 7
Implementation 7
Programming paradigms 8
Classification of languages 8
Programming style 8
Paradigms in languages 8
Pure paradigms 8
Multiparadigms languages 9
Main paradigms 10
Imperative programming paradigm 10
Derived paradigms 10
Languages that implement it 10
Characteristics of imperative language 10
Declarative programming paradigm 11
Derived paradigms 11
Languages that implement it 11
Event-driven programming paradigm 12
Implementation cases 12
Languages that implement it 12
Structured programming paradigm 13
Languages that implement it 13
Object-oriented programming paradigm 14
Languages that implement it: 14
Objects 15
Classes 15
Properties 15
Methods 15
Inheritance 15
Encapsulation 15
Abstraction 16
Polymorphism 16
Advantages of object-oriented programming 16

Disadvantages of object-oriented programming 16
Procedural programming paradigm 17
Functional programming paradigm 18
Higher-order functions can: 18
Languages that implement it: 18
Functions 19
Characteristics of pure functions: 19
Side effects 20
These side effects are observable changes: 20
Logic programming paradigm 21
Areas of Application: 21
Concurrent programming paradigm 22
Created 23
Executable 23
Running 23
Blocked 23
Finished 23
Concurrency issues 25
Traffic lights. 25
Monitors 25
- Initialization: 26
- Private Data: 26
- Monitor Methods: 26
- Entry Queue: 26
The 1940s: the first computers and assembler languages 27
The 1950s 28
Why are there so many programming languages? 28
Fortran 29
ALGOL 29
The 1960s 30
LISP 30
COBOL 31
Procedures 31
Data 31
Environment 31
Identification 31
Simula 32
The 1970s 33
C 33
Pascal 34
Smalltalk 34

ML 35
Prolog 35
The 1980s 36
C++ 36
Python 37
The 1990s 38
Java 38
JVM 38
JavaScript 39

Abstract machine

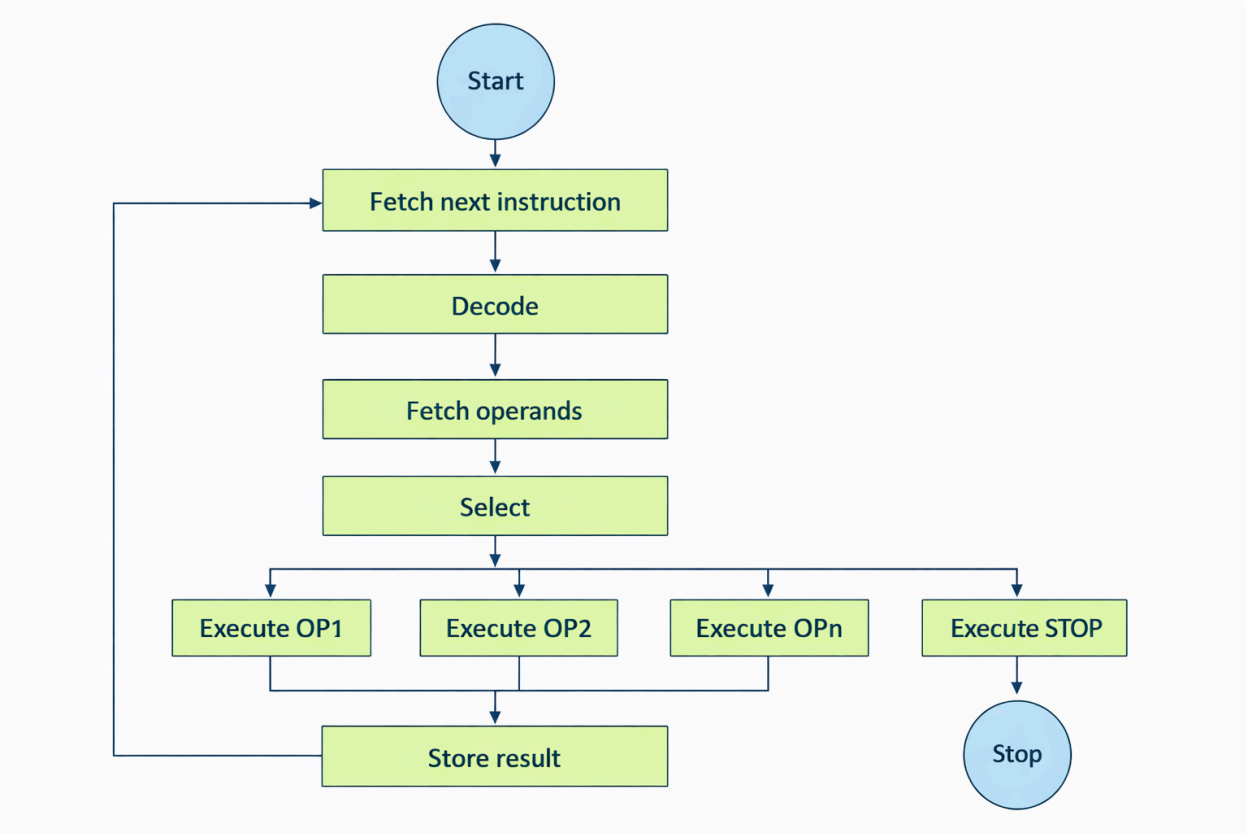
An abstract machine refers to a computer at a conceptual or theoretical level. This computer executes algorithms that process instructions in a programming language using data types and data structures.



An abstract machine can perform the four basic operations:

- Process primitive data (integers, floats, booleans, etc.).
- Control execution sequence (cycles, conditions, loops, etc.).
- Control data transfer.
- Memory management.

Execution cycle



Programming languages vs natural languages

An abstract machine works with a programming language.

A language lets people communicate:

- Express ideas.
- Express feelings.
- Give instructions.
- Achieve a goal collaboratively.

Computers use a programming language to give instructions to the machine. Some languages even allow for a high level of specificity with these instructions.

Therefore, it is said that:

> *"Computers do not make mistakes; rather, people give bad instructions, and the computer only does what it was told to do."*

A language is:

> *"Each person's individual style and manner of speaking and writing."*

Programming languages are an artificial, formal system for expressing algorithms.

Levels of description

Natural and programming languages are divided into three levels of description.

Grammar

Which sentences are correct?

- The sequence of symbols forms words | tokens.
- Syntax allows us to form sentences in the language itself.

Semantics

What does a sentence mean?

- While loops.
- Variables.
- Functions.
- Classes.

Pragmatics

How to use a meaningful sentence?

- Meanings to be interpreted.

Pragmatics can be interpreted differently by different people. Therefore, understanding it is very important for programming.

Implementation

How does the computer transform code into instructions?

Programming paradigms

> *“Theory or set of theories whose core is accepted without question and which provides the basis and/or model for solving problems and advancing knowledge.”*

In programming, a paradigm refers to the

Classification of languages

Based on their characteristics. There are multi-paradigm languages.

Programming style

By the way it approaches a problem or the tools used.

Paradigms in languages

Programming languages can be classified according to their paradigm.

Pure paradigms

- Smalltalk -> POO.
- Haskell -> Functional.

Multiparadigms languages

- C++.
- Java.
- JavaScript.
- Scala.
- PHP.
- Python.
- Ruby.
- F#.
- etc.

For a programming language to work with a specific paradigm means:

- Restrictions or limitations.
- Side effects.
- Use of GoTo.

Programming languages can be divided into:

- Interpreted.
- Compiled.

Main paradigms

These are the concepts that a programming language can work with.

Today, there are more than 100 programming paradigms. The most popular and widely used in the job market.

Imperative programming paradigm

These are instructions focused on the step-by-step process for performing a task.

Therefore, it is normal for the code to be structured in sequence or for there to be some control structures that indicate how the instructions are carried out.

Imperative programming is the most popular and widely used paradigm worldwide.

Derived paradigms

- Procedural.
- Object-oriented.

Languages that implement it

- Fortran.
- Java.
- C.
- Python.
- PHP.
- Ruby.
- Etc.

Characteristics of imperative language

- Control structures.
- Input/Output.
- Error and exception handling.
- Procedure abstraction.
- Expression and assignment.
- Support for data structures.

Declarative programming paradigm

This paradigm is the opposite of the imperative paradigm. It focuses on the result rather than the step-by-step process of how it is achieved. The aim is to reduce or eliminate side effects.

A side effect is the result or effect that calling a function can have on a data structure, variables, or another function.

Derived paradigms

- Functional.
- Logical.
- Mathematical.
- Reactive.

Languages that implement it

- Prolog.
- LISP.
- Haskell.
- SQL.
- Elixir.
- XPath.
- Regex.
- etc.

Event-driven programming paradigm

The event-driven programming paradigm is characterized by the fact that it does not control the sequence of execution but reacts to events that occur. These events are perceived by event handlers.

When the program runs indefinitely, these events can occur, generating a series of actions or procedures.

Types of events

- Clicks.
- Keys.
- Sensors.
- Messages.
- Triggers.

Implementation cases

- GUIs.
- WEB applications.
- Booking systems.
- Alarm systems.
- Robotics.
- Video games.

Languages that implement it

- Java.
- JavaScript.
- C#.
- GUI libraries/frameworks.
- Any object-oriented language.

Structured programming paradigm

It emerged in the mid 1940s, when Von Neumann's architecture recognized that a machine, a computer, could store both a program and the data generated by it.

This concept is quite similar to the abstract machine.

Structured programming can make use of control structures. In 1968, Dijkstra warned of the damage caused by the GoTo statement.

Try to avoid using the GoTo statement. The GoTo statement is an instruction that was used in programs to indicate that the code would jump from one line to another. This makes it very complex, since if you try to visualize or graph the sequence in which a GoTo sends one line to another at another point. This would result in a large number of lines, which makes it very complex. This is solved with control structures.

Languages that implement it

- Fortran.
- Pascal.
- Cobol.
- Ada.
- Perl.
- C.
- Etc.

Object-oriented programming paradigm

Procedural programming solved many problems of imperative programming.

When there are programs with hundreds of procedures that must be called, the lines of code are executed, the procedure is exited, and a new one is entered. This continues until the program's objective is completed. This makes it very complex.

Today, web and mobile applications have multiple action buttons that execute code when the user interacts with them. This would be extremely complex through procedural programming if it depended on a main procedure.

In this context, object-oriented programming emerges.

It contains data and code:

- Data in the form of fields (attributes or properties).
- Code in the form of procedures (methods).

It originated in the late 1950s when objects were discussed at MIT (Massachusetts Institute of Technology) and later cited by Alan Kay.

> “I thought of objects as biological cells and/or individual computers in a network, capable of communicating with messages.”

Object-oriented programming is the most widely used paradigm today and around the world.

Languages that implement it:

- Simula.
- Cobol.
- Java.
- C++.
- C#.
- Python.
- Ruby.
- PHP.
- etc.

Simula was the first language to implement this paradigm. It introduced concepts such as:

- Classes.
- Objects.
- Inheritance.
- Dynamic links.

Objects

An object is a template that represents a real-life object.

Classes

The different classes or instances that arise from the object.

Properties

Each class has properties or attributes. These can be shared characteristics that differentiate each one, making it unique.

Methods

Methods or behaviors are specific actions performed by classes.

Object-oriented programming is based on four essential pillars:

- Inheritance.
- Encapsulation.
- Abstraction.
- Polymorphism.

Inheritance

Allows a class to have a subclass that inherits attributes from it.

Encapsulation

Allows part of a class's code to be isolated.

Abstraction

Allows the characteristics of an object to be isolated.

Polymorphism

Allows calls from one class to be interpreted differently from another class.

Advantages of object-oriented programming

- Reusable code.
- Reduced redundancy.
- Easy to maintain.
- Security.

Disadvantages of object-oriented programming

- Size.
- Effort.
- Speed.

Procedural programming paradigm

The procedural programming paradigm is derived from structured programming, which solved major problems with its characteristics and implementations.

This paradigm groups code into blocks of procedures. These are instructions that the program will continue to execute, giving the program a certain order and structure. It works with a stack system, as it is sequential, where a procedure in one of its lines calls another procedure, and there may be more calls to other procedures before returning to the main program.

It shares similarities with the object-oriented programming paradigm.

Procedural	Object-Oriented
Procedure	Method
Record	Object
Module	Class
Procedure Call	Message

Languages that implement it:

- Fortran.
- Algol.
- Cobol.
- Basic.
- etc.

Functional programming paradigm

This paradigm corresponds to the declarative paradigm in programming.

It is considered a complex paradigm to understand, analyze, and implement.

This functional paradigm allows for the creation of higher-order functions. Furthermore, functional programming prioritizes the use of recursion and higher-order functions to solve problems.

Higher-order functions can:

- Take one or more functions as parameters.
- Return a function as output.

It aims to minimize or avoid side effects. These arise when one function affects another.

It originated in the 1930s when Alonzo Church developed lambda calculus. In the same decade, Alan Turing demonstrated that Turing machines are equivalent to lambda calculus; consequently, there was a certain similarity that allowed the concept to be applied to programming.

The functional programming paradigm emerged in the late 1950s with the development of the LISP programming language, which implemented Church's lambda notation.

Languages that implement it:

- LISP.
- Scheme.
- Clojure.
- Racket.
- Erlang.
- Haskell.
- f#.
- Python.
- etc.

Functions

"A relationship between two sets that assigns to each element of the first set an element of the second set or none."

In programming, a function is a type of data or object that takes a value "X" and produces a value "Y".

It may or may not return a result.

$$f(x) = X + 3$$

$$f(x) = x^2 + 3$$

Characteristics of pure functions:

- They are deterministic.
- A single input value produces a single output value.
- They do not produce side effects.
- They do not include impure functions.

Characteristics of lambda functions:

- Anonymous function.
- Single-use behavior.
- Specific or short-lived context.
- Simplicity.

Side effects

It is common to talk about side effects in imperative programming, particularly when working with functions or variables that are in the global scope or have a defined scope.

It is important to define the scope of a variable; this ensures that code within one function does not affect another function.

> *"The way someone or something behaves in response to a particular stimulus."*

Functions, variables, data, files, and programs can be affected by external factors from other data or data structures; that is why functional programming avoids them.

These side effects are observable changes:

- Application status.
- Data values.
- File modifications.
- Server requests.
- Etc.

Logic programming paradigm

It is a paradigm that expresses objectives as a set of statements or rules regarding outcomes and constraints in mathematical logic.

Areas of Application:

- NLP.
- Information retrieval in databases.
- Mathematical applications.

The paradigm is based on Horn clauses, which are propositions defined by predicates.

```
h = Fact | Head of the rule
p = Predicate | Body
snowing(C) <- rain(C), cold(C)
```

This paradigm originated in the 1960s when Cordell Green proposed the use of clauses in programs.

Prolog was the language that spurred the use of this paradigm.

This paradigm is used more in academic and research settings than in today's industry.

To speak of logic programming is also to speak of mathematics and the evaluation of these expressions.

Languages that implement it:

- Prolog.
- ALF.
- Fril.
- Mercury.
- Oz.
- Visual Prolog.
- XSB.
- etc.

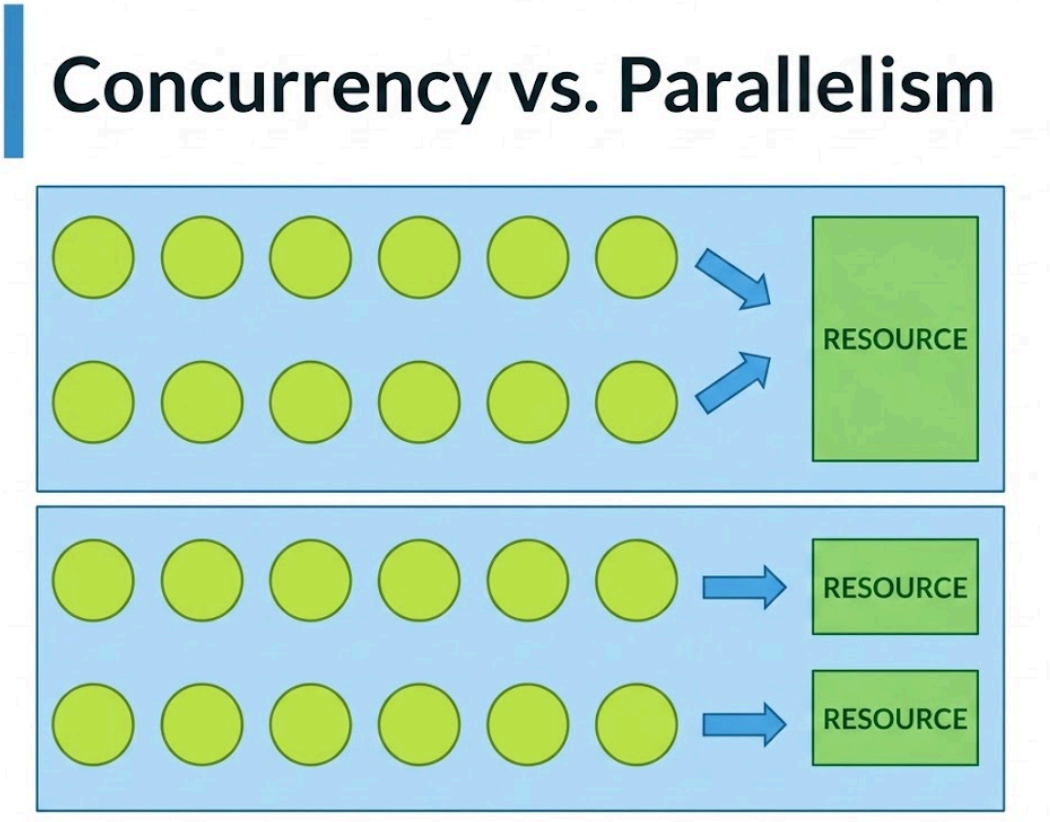
Concurrent programming paradigm

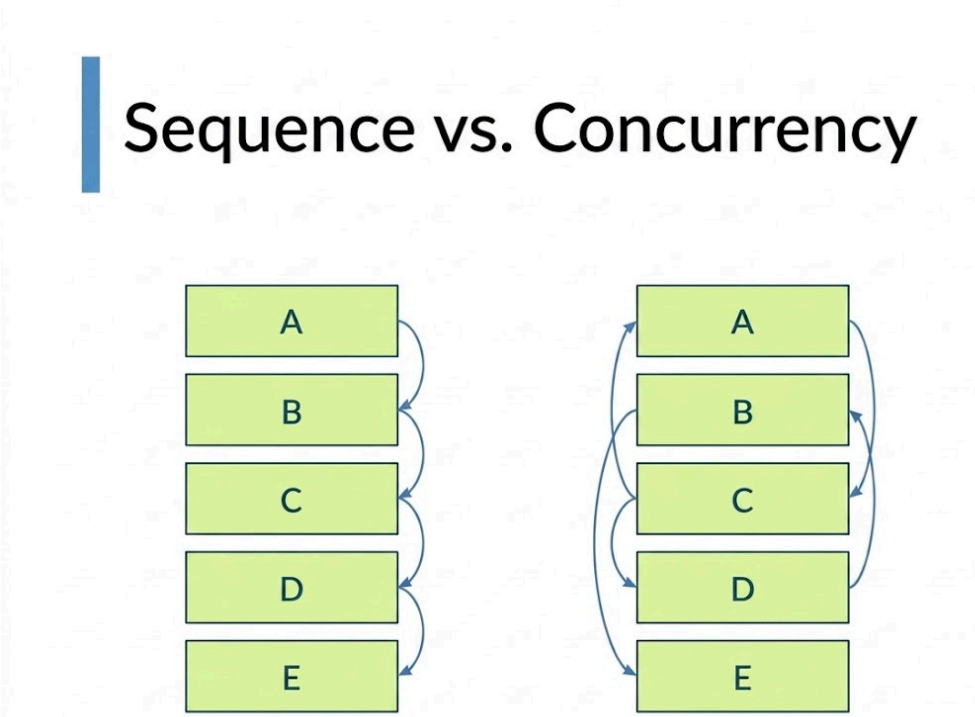
Concurrent programming is very important in computer science.

Parallelism in computing involves dividing multiple tasks among available resources.

"Coincidence: the simultaneous occurrence of several circumstances."

In programming, it refers to the ability of a program or algorithm to have two or more separate execution contexts, whether through preprocessors or threads.





Thread States

Created

Not ready to run.

Executable

Waiting to be executed.

Running

It is running on the processor.

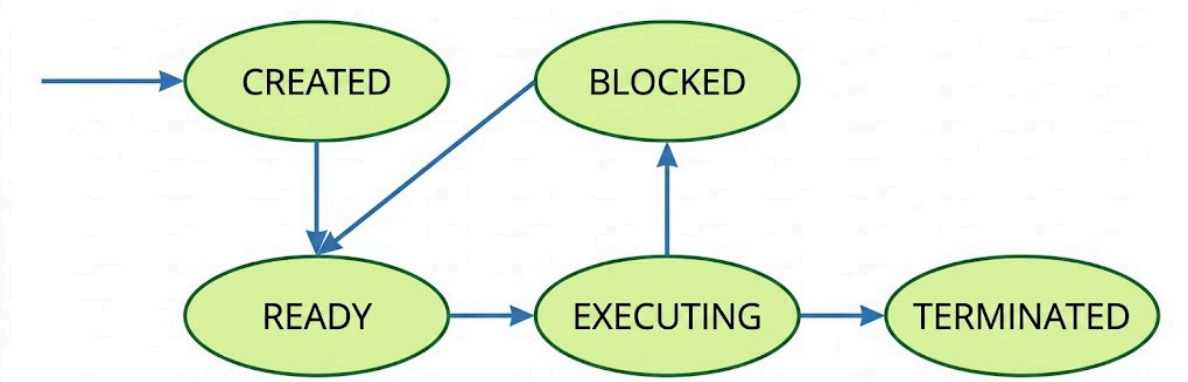
Blocked

It is waiting to enter the code that requires access to the shared resource or is exiting the process.

Finished

It has stopped and cannot be restarted.

Codecamp academy



A critical section in a thread is a section of code that requires exclusive access to a shared variable.

Inter-thread Communication:

- Exclusive access to a shared resource.
- Exchanging information with another thread.

Concurrency issues

One of the concurrency issues is “critical races”: these occur when two or more threads attempt to access a resource as quickly as possible.

Another problem with concurrency is “deadlocks,” which occur when a thread waits indefinitely for access to a resource needed to execute its task, since it can never gain access; as a result, the tasks are not executed, and this becomes a problem.

For a stalemate to occur, four conditions must be met:

1. Threads must have exclusive access to resources.
2. Threads must hold some resources while waiting for others.
3. Resources cannot be removed from waiting threads.
4. There is a circular chain of threads, each of which holds one or more resources from the next thread.

There are various strategies for avoiding deadlocks.

Preventing certain conditions from being met:

- Traffic lights.
- Monitors.

Traffic lights.

These are integer variables associated with a thread queue mechanism. The use of semaphores through state variables is very similar to the event-driven programming paradigm; in semaphores, this is done at the thread level.

Monitors

They are a kernel-based data structure in operating systems that consists of four main components.

- **Initialization:**

  Contains code to be executed.

- **Private Data:**

  Procedures used from within the monitor.

- **Monitor Methods:**

  Procedures that can be called from outside.

- **Entry Queue:**

  Threads that call a monitor method but do not have permission to run.

Concurrency can be handled differently from one programming language to another. That is why it is important to familiarize yourself with the programming language's documentation, read about how that language handles concurrency, and thus be able to resolve any problems that arise when implementing the language.

Languages that implement it:

- JavaScript.
- C#.
- Go.
- Rush.
- Haskell.
- Elixir.
- etc.

The 1940s: the first computers and assembler languages

The first computer was not an abstract machine, since that is a theoretical concept.

There are different criteria for defining which was the first computer; some authors say it was the EDSAC (Electronic Delay Storage Automatic Calculator), since it met the characteristics of the von Neumann architecture.

- Electronic and digital.
- Performs 4 basic operations.
- It is programmable.
- It stores programs and data.

The EDSAC was inspired by the EDVAC, as it implemented some of von Neumann's ideas.

The MARK I and ENIAC computers lacked the linguistic tools to be programmed at the level of the EDSAC, since the EDSAC could be programmed using binary code.

Throughout history, computers have used various methods of data input, such as:

- Switches.
- Knobs.
- Light bulbs.
- Cables.
- Etc.

These allowed communication with computers using binary code and are known as 1GL (First-Generation Languages).

Later, a set of languages known as assembly languages emerged. They are considered a set of languages because each one was limited by the characteristics of the computer's hardware, which meant that different models or types of computers used different assembly languages—making program portability impossible.

Assembly language is a symbolic representation of binary code. This paved the way for the next generation of programming languages (2GL, or Second Generation Languages), as the evolution of these languages made program portability possible; the first devices that enabled program portability were punch cards.

The 1950s

The first high-level programming languages appeared in the 1950s. These are characterized by:

- Ignore hardware limitations.
- Have reliable programs.
- The syntax resembles human language.

2GLs (Second-Generation Languages) share the same characteristics as natural language.

There are currently more than 100 programming languages.

Why are there so many programming languages?

- Hardware.
- Specific applications.
- New methodologies.
- Implementations.
- Theories.

Fortran

Fortran was the first programming language created in this decade, in 1957. Fortran (FORmula TRANsformation) was the first high-level imperative language developed by John Backus and his colleagues. It was primarily used in numerical scientific applications on the IBM 704 computer.

Subsequent versions include:

- Control Structures.
- Functions.
- Dynamic Memory.
- Comments.
- Main Routine with Subprograms.

ALGOL

In 1960, ALGOL 60 was defined, and the development committee remained in place—that is, it remained the same until 1968, when ALGOLW was released.

ALGOL 58–60 brought programming closer to mathematics; that is, the notation used to write the code closely resembles mathematical notation.

ALGOL's Contributions to Programming:

- Passing parameters by name.
- Block structures.
- Generative grammar.

The 1960s

Following the emergence of the first programming languages in the 1950s, it was common for languages to evolve to accommodate new computers or for new programming languages to emerge for specific applications and implementations.

LISP

LISP (LISt Processor) was designed by John McCarthy's team at MIT and is based on lambda calculus.

It was one of the first programming languages whose applications were not numerical.

Purposes of LISP:

- Manipulate S-expressions.
- Automatically translate texts.

Initially, it had to be adapted to different architectures, computers, platforms, and languages. As a result, it was not very efficient as a programming language.

Computers capable of running the LISP language were called LISP machines.

Contributions:

- High-level programming.
- Dynamic memory.

Common Dialects:

- Racket.
- Common Lisp.
- Scheme.
- Clojure.

COBOL

COBOL (Common Business Oriented Language) is a business-oriented programming language based on the:

- Imperative.
- Procedural.
- Object-oriented.

One of its features is that it aims to resemble human natural language as closely as possible, specifically the English language.

In 1968, it was revised by Grace Hopper's team, making it a standard.

COBOL is characterized by dividing programs into four sections

Procedures

Algorithmic aspect code.

Data

Description of the data.

Environment

Specification of the environment external to the program, indicating the physical machine.

Identification

Program name, author, etc.

Simula

This programming language gained prominence in the 1960s; it was derived from ALGOL 60 and designed in 1962.

The purpose of Simula is to perform simulation calculations involving queues. Various factors can be added to these queues that will make calculating service time more complex.

Simula is the predecessor of Smalltalk and C++.

It introduced concepts such as:

- Class.
- Object.
- Subtype.
- Dynamic dispatch method.

The 1970s

In the 1970s, along with new programming languages and paradigms, the microprocessor emerged, leading computing to where it is today. This meant that computers became more accurate, faster, and smaller. It also meant that programming languages had to adapt to this new architecture, which allowed programs to have capabilities that were far more revolutionary than those of previous decades.

C

The C programming language was designed by Dennis Ritchie and Ken Thompson at AT&T Bell Laboratories in 1972.

Its original purpose was to write programs for Unix operating systems.

Its name derives from the fact that it is the successor to the B programming language, a simplified version of BCPL (Basic Combined Programming Language).

Key features:

- More opportunities for low-level access.
- Portable.
- Program interactive systems.
- Process data in real time.
- Compile to efficient machine code.
- Suitable for system programming and general-purpose use.

In modern systems such as web and mobile programming, the C language is not used directly, but a programming language is used that adds a layer of abstraction involving an interpreter, and it is very likely that this interpreter uses the C language.

As it continues to evolve and improve, it allows for programming:

- Microcontrollers.
- Embedded systems.
- Robotics.
- Home automation.
- Electronics.

Pascal

The language was developed by Niklaus Wirth as a simplification of AlgolW.

What made it popular during this decade was that it was the most widely used language for educational purposes in the 1980s.

PASCAL stood out for introducing the concept of intermediate code

The PASCAL language shares characteristics with the C language, such as:

- Static typing.
- Code blocks.

Smalltalk

The Smalltalk programming language was developed in the 1970s by Alan Kay at Xerox PARC.

This programming language includes built-in objects; in other words, there is no need to import them from a library or framework.

Its main feature was its ability to handle concepts such as:

- Objects.
- Classes.
- Encapsulation.
- Public methods.
- Private instances.

It also sought to provide a comprehensive system for programming.

- Language.
- Programming environment.
- A machine designed for efficiency.
- Object-oriented.

ML

The ML (Meta Language) language was developed by Robin Milner and his colleagues in the mid-1970s.

Its general purpose is the manipulation of symbolic information. It is more research-oriented.

Like LISP, this language follows a functional paradigm. ML incorporates some of PASCAL's safety features into a statically typed system.

It was the first language to feature a type inference system; that is, even without explicitly specifying the data type, it could infer the data type to which a character belonged.

Prolog

Prolog was the first language to implement the logic paradigm; moreover, it is still in use today.

The language is influenced by the theory of automatic deduction.

It was developed by Alain Colmerauer and Philippe Roussel based on Robert Kowalski's Horn clauses.

Its first implementation was an interpreter written in Fortran.

The 1980s

The 1980s also saw the emergence of personal computers; they were much more accessible to the general public.

These computers introduced a different way for users to interact with the computer. They did so through GUIs, which meant that programming languages were geared toward this style of programming and, therefore, toward a paradigm different from the one used decades earlier.

C++

This programming language was designed by Bjarne Stroustrup in 1986 as an extension of C. However, it evolved into a standalone programming language.

The use of classes is the main difference from the C language. Its syntax is similar to that of C.

A program written in C can be compiled by a C++ compiler, meaning there is some compatibility.

- It improved its type system.
- It implemented templates.
- It manipulates objects directly.
- It improved the inheritance system.

This makes it a more effective language than C.

Python

It is a general-purpose programming language created in the late 1980s and released in 1991 by Guido Van Rossum.

It is currently one of the most popular languages in the world, along with Java and C.

Features:

- Interpreted.
- High-level.
- General-purpose.
- Multiparadigm.
- Includes multiple components.

Areas of Implementation:

- Web Development.
- IoT.
- Mobile.
- Video Games.
- Servers.
- AI.
- Data Science.
- GUI.

The 1990s

The 1990s saw the emergence of web programming, which was one of the milestones that transformed programming and shaped its evolution into what it is today. This led to the emergence of new languages designed exclusively for the web.

- Ruby.
- PHP.

Languages designed for different purposes also emerged:

- R.
- Haskell.

Java

The programming language was developed by the Green Team, led by Jim Gosling at Sun Microsystems in the early 1990s.

It is intended for use on network-connected devices with limited computing power.

Its potential on the Web lies in the use of applets. These are now obsolete.

JVM

The JVM (Java Virtual Machine) allows application code to be compiled into an intermediate form, known as bytecode. It also converts the bytecode into machine code so that it can be executed.

Java is a programming language that allows for a certain level of low-level access, though not as specific as C.

In 1995, the JVM was incorporated into the Netscape web browser.

Security in Java:

- It does not generate undetected runtime errors.
- Type checking, compilation, and bytecode verification.

This ensures that Java programs are secure and cannot be modified once the bytecode is revealed.

Key Features:

- Simplicity of language.
- Garbage collector.
- Implicit pointer handling.
- Use of threads for concurrency.

JavaScript

It is a high-level, just-in-time compiled, multi-paradigm, weakly typed, dynamically typed, and prototype-based programming language.

Its main feature of the object-oriented paradigm is prototypical inheritance.

It is also a programming language used on both the client and server sides.

As of 2012, it is supported by all web browsers by default, as it has become a standard web technology.

It supports paradigms:

- Essential.
- Functional.
- Designed for events.